

CHAPTER 4

IMPLEMENTATION AND MAINTENANCE

4.1 INTRODUCTION

The Implementation phase involves translating the design specifications from Chapter 3 into actual working code. The Job Portal was developed using the MERN stack — MongoDB, Express.js, React.js, and Node.js — with Cloudinary for cloud-based resume storage and JWT for role-based authentication. The project is split into two directories: /frontend (React + Vite, running on port 5173) and /backend (Node.js + Express.js, running on port 8000).

4.2 PROJECT FOLDER STRUCTURE

The repository is organised into two main directories. The backend directory (backend/src/) contains the following subdirectories:

- controllers/ — Route handler logic files: application.controller.js, candidate.controller.js, company.controller.js, job.controller.js, saved.controller.js, user.controller.js
- db/ — Database connection: db.js
- middlewares/ — adminPanel.js (recruiter role check), asyncHandler.js (async error wrapper), authMiddleware.js (JWT verification), cloudanry.js (Cloudinary config)
- models/ — Mongoose schemas: application.models.js, candidte.models.js, company.models.js, job.models.js, saved.models.js, user.models.js
- routes/ — Express route files: application.routes.js, candidate.routes.js, company.routes.js, job.routes.js, save.routes.js, user.route.js
- utils/ — cloudnaryConfig.js, multer.js
- app.js — Express app configuration, server.js — Server entry point

The frontend directory (frontend/src/) contains:

- api/api.js — Axios base configuration for all API requests
- components/ — JobId.jsx, Nav.jsx — shared reusable components
- pages/ — Subdirectories: candiadate/, company/, job/, profileSection/, user/ — plus Home.jsx, Login.jsx, Register.jsx
- routes/MainRouters.jsx — React Router DOM configuration
- styles/ — Individual CSS files for each page: Application.css, CandidateAll.css, CompanyJobs.css, CreateCompany.css, CreateJob.css, EditAll.css, EditJob.css, Home.css, Job.css, JobId.css, LoginRegister.css, Myjob.css, Nav.css, Profile.css, SavedJobs.css
- App.jsx, main.jsx, index.css — Root component, entry point, global styles

4.3 MODULE-WISE IMPLEMENTATION

The backend is built using a clean MVC pattern. Each route file maps HTTP endpoints to controller functions. An asyncHandler utility wraps all async controllers to automatically forward errors to Express's error handler, eliminating repetitive try-catch blocks across modules.

4.3.1 Module 1 – User Registration and Login (JWT Auth)

Both Job Seekers and Recruiters register through the same endpoint. The role field passed at registration determines what features they can access after login. The registerHandler performs: input validation (username, password, email presence check), phone number validation (exactly 10 digits via regex), duplicate email check (returns HTTP 409 if email already registered), password hashing with bcrypt (salt factor 10), user creation in MongoDB, JWT token signing with 5-minute expiry, and setting the token as an HTTP-only Secure SameSite=None cookie. The function returns HTTP 201 on success with the new user's details.

The following code shows the complete registerHandler implementation from user.controller.js:

```
const registerHandler = asyncHandler(async(req,res)=>{
  let {username, password, email, phoneNumber} = req.body

  // check all fields required or not
  if(!username.trim() || !password.trim() || !email.trim()){
    return res.status(400).json({
      message: "username , password , email is required"
    })
  }
  // check if phone number is equal to 10 digits or not
  if (!phoneNumber || !/^\d{10}$/.test(phoneNumber)) {
    return res.status(400).json({
      message: "Phone number must be exactly 10 digits"
    });
  }

  email = email.toLowerCase().trim();

  // check user already exists
  const checkUser = await userModel.findOne({email})
  if(checkUser){
    return res.status(409).json({ message: "Email already exist" })
  }

  // hash password
  const hashPassword = await bcrypt.hash(password, 10);

  const newUser = await userModel.create({
    username,
    password: hashPassword,
    email,
    phoneNumber,
  })

  const token = jwt.sign({id: newUser._id},
    process.env.JWT_SECRET, {expiresIn: "5m"})

  res.cookie("token", token, {
    httpOnly: true,
    secure: true,
    sameSite: "None"
  });

  return res.status(201).json({
    message: "User is created succesfully",
    user: {
      username: newUser.username,
      email: newUser.email,
      phoneNumber: newUser.phoneNumber
    },
    token
  })
})
```

```
    })  
  })
```

4.3.2 Module 2 – JWT Authentication Middleware (authMiddleware.js)

The authMiddleware is applied on all protected routes. It reads the JWT token from the browser cookie (req.cookies.token), returns HTTP 401 if no token is present, verifies the token using jwt.verify() with the JWT_SECRET, attaches the decoded user ID to req.user, and calls next() to pass control to the route controller. Any verification error (expired or invalid token) returns HTTP 400.

The following code shows the complete authMiddleware implementation:

```
const jwt = require('jsonwebtoken')  
  
async function authMiddleware(req, res, next){  
  try {  
    const token = req.cookies.token  
  
    if(!token){  
      return res.status(401).json({ message: "token is  
missing" })  
    }  
  
    const decode = jwt.verify(token,  
process.env.JWT_SECRET);  
    req.user = decode.id  
    next();  
  
  } catch (error) {  
    res.status(400).json({ message: "token is invalid" })  
  }  
}  
  
module.exports = authMiddleware
```

4.3.3 Module 3 – Recruiter Role Middleware (adminPanel.js)

The recruiterPanel middleware is stacked after authMiddleware on all recruiter-only routes such as job creation and applicant management. It queries MongoDB for the current user and checks whether their role field equals 'recruiter'. If not, it returns HTTP

400 with an access denied message. Only verified recruiters can call next() and proceed to the controller.

The following code shows the complete recuriterPanel middleware:

```
const userModel = require('../models/user.models')
const asyncHandler = require('./asyncHandler')

const recuriterPanel = asyncHandler(async(req, res, next)=>{
  const user = req.user

  const currentUser = await
  userModel.findById(user).select("role")

  if(!currentUser){
    return res.status(401).json({ message: "user is not
    avaiable" })
  }

  if(currentUser.role !== "recruiter"){
    return res.status(400).json({ message: "your not
    recruiter not allowed!" })
  }

  next();
})

module.exports = recuriterPanel
```

4.3.4 Module 4 – Cloudinary Configuration (cloudanry.js)

Cloudinary is integrated as the cloud storage solution for all resume file uploads. The configuration module initialises the Cloudinary SDK v2 using credentials stored in the .env file. This module is imported wherever file upload or retrieval is needed, keeping credentials centralised.

```
const cloudnary = require("cloudinary").v2;

cloudnary.config({
  cloud_name: process.env.cloud_name,
  api_key: process.env.api_key,
  api_secret: process.env.api_secret,
});
```

```
module.exports = cloudnary;
```

4.3.5 Module 5 – Create Job Controller (job.controller.js)

The createJobController handles new job posting by recruiters. Before creating a job, it verifies the recruiter is associated with the specified company by querying companyModel with both the companyId and userId. It then validates required fields (title, description, location, salaryRange) and validates that requirement is a non-empty array before creating the job document in MongoDB.

```
const createJobController = asyncHandler(async(req, res)=>{
  const userId = req.user
  const {companyId} = req.params

  const recruiterInCompany = await companyModel.findOne(
    { _id: companyId, createdAt: userId }
  )

  if(!recruiterInCompany){
    return res.status(400).json({ message: "you are not
create job" })
  }

  const {title, description, requirement, jobType, workMode,
location, experienceLevel, salaryRange, statusNow} =
req.body

  if(!title || !description || !location || !salaryRange){
    return res.status(400).json({ message: "all filed
compalasy" })
  }
  if(!requirement || !Array.isArray(requirement) ||
requirement.length === 0){
    return res.status(400).json({ message: "requirement is
must be not empty" })
  }

  const jobForm = await jobModel.create({
    companyId,
    recruiterId: userId,
    title, description, requirement,
    jobType, workMode, location,
    experienceLevel, salaryRange, statusNow
  })

  return res.status(201).json({ message: "job created
successfully", jobForm })
})
```

4.3.6 Module 6 – Job Search and Filter (job.controller.js)

The searchJobs function provides server-side filtering of job listings. It accepts optional keyword and location query parameters and builds a MongoDB filter object dynamically using case-insensitive regex (\$regex with \$options: 'i'). Results are sorted newest-first and populated with company details via .populate('companyId').

```
const searchJobs = async (req, res) => {
  try {
    const { keyword, location } = req.query;
    let filter = {};

    if (keyword) {
      filter.title = { $regex: keyword, $options: "i" };
    }
    if (location) {
      filter.location = { $regex: location, $options: "i" };
    }

    const jobs = await jobModel.find(filter)
      .populate("companyId")
      .sort({ createdAt: -1 });

    res.json({ jobs });
  } catch (err) {
    res.status(500).json({ message: "Search failed" });
  }
};
```

4.3.7 Module 7 – Save Job Feature (saved.controller.js)

The savedController allows job seekers to bookmark jobs. It checks if a save record already exists for the userId–jobId pair to prevent duplicates, then creates a new save document. The allSavedJobController retrieves all saved jobs for the logged-in user with full job details via .populate('jobId').

savedController — Save a Job:

```
const savedController = asyncHandler(async (req, res) => {
  const userId = req.user
  const { jobId } = req.params

  const saveJob = await saveModel.findOne({ userId, jobId })
```

```

        if(saveJob){
            return res.status(400).json({ message: "saved job
already exist" })
        }
        const job = await saveModel.create({ userId, jobId })
        return res.status(200).json({ message: "saved job is
create", job })
    })

```

allSavedJobController — Retrieve All Saved Jobs:

```

const allSavedJobController = asyncHandler(async(req, res)=>{
    const userId = req.user

    const savedJob = await
saveModel.find({userId}).populate("jobId")

    if(!savedJob || savedJob.length === 0){
        return res.status(400).json({ message: "no jobs
avaible" })
    }
    return res.status(200).json({ message: "all saved jobs",
savedJob })
})

```

4.3.8 Module 8 – Apply for Job with Resume Upload (application.controller.js)

The applicationController handles job application submission. It enforces a duplicate-application check, validates that a resume file is attached, converts the file buffer to a data URI using `bufferToDataUri()`, uploads it to Cloudinary's 'jobportal/resumes' folder with resource_type: 'raw' (required for PDF support), and stores the resulting secure URL, public ID, and filename in the application document.

```

const applicationController = asyncHandler(async(req, res)=>{
    const {jobId} = req.params
    const userId = req.user

    const alreadyApplied = await
applicationModel.findOne({jobId, userId})
    if(alreadyApplied){
        return res.status(409).json({ message: "user is already
applied" })
    }

    if (!req.file) {

```



```

        return res.status(400).json({ message: "Resume file is
        required" });
    }

    const dataUri = bufferToDataUri(req.file);

    const result = await cloudinary.uploader.upload(dataUri, {
        folder: "jobportal/resumes",
        resource_type: "raw",    // IMPORTANT for PDF
    });

    const application = await applicationModel.create({
        jobId,
        userId,
        statusNow: "applied",
        resume: {
            url: result.secure_url,
            publicId: result.public_id,
            fileName: req.file.originalname,
        }
    })

    return res.status(201).json({ message: "applied
    succesully", application })
})

```

4.3.9 Module 9 – Check Application Status (application.controller.js)

The checkAppliedController provides a lightweight boolean check that the frontend uses to determine whether to show 'Apply Now' or 'Already Applied' on the job detail page. It uses the double-negation operator (!!) to convert the findOne() result to a strict boolean value.

```

const checkAppliedController = asyncHandler(async (req, res) =>
{
    const userId = req.user;
    const { jobId } = req.params;

    const alreadyApplied = await applicationModel.findOne({
        userId, jobId });

    res.status(200).json({ applied: !!alreadyApplied });
});

```

4.4 API ENDPOINT SUMMARY

The backend exposes the following REST API endpoints, all prefixed with `http://localhost:8000/api/v1`:

Method	Endpoint	Auth	Description
POST	/user/register	No	Register new user (seeker/recruiter)
POST	/user/login	No	Login – receive JWT cookie
GET	/user/logout	Yes	Clear JWT cookie
PUT	/user/profile/update	Yes	Update profile + upload resume
POST	/user/save/:jobId	Yes	Save or unsave a job
POST	/job/post/:companyId	Recruiter	Create new job posting
GET	/job/get	No	Get all jobs (supports ?keyword= ?location=)
GET	/job/get/:id	No	Get single job details
PUT	/job/update/:id	Recruiter	Edit existing job
DELETE	/job/delete/:id	Recruiter	Delete a job posting
POST	/application/apply/:jobId	Seeker	Apply for a job with resume
GET	/application/get	Seeker	Get all applications of logged-in seeker
GET	/application/check/:jobId	Seeker	Check if seeker already applied
GET	/application/:jobId/applicants	Recruiter	Get all applicants for a job
PUT	/application/status/:id	Recruiter	Accept or Reject an application
GET	/saved/all	Seeker	Get all saved jobs
POST	/saved/:jobId	Seeker	Save a specific job

4.5 ENVIRONMENT CONFIGURATION

The backend requires a `.env` file in the `/backend` directory with the following environment variables:

```
PORT=8000
MONGO_URI=mongodb+srv://<username>:<password>@cluster.mongodb.net/jobportal
JWT_SECRET=your_jwt_secret_key
JWT_EXPIRY=5m
cloud_name=your_cloudinary_cloud_name
api_key=your_cloudinary_api_key
api_secret=your_cloudinary_api_secret
```

4.6 MAINTENANCE

4.6.1 Corrective Maintenance

Corrective maintenance involves fixing bugs identified during or after testing. Examples include: edge cases in the Accept/Reject application flow, incorrect status badge rendering on the frontend, and validation gaps in the registration handler that allowed malformed phone numbers.

4.6.2 Adaptive Maintenance

Adaptive maintenance involves updating environment configurations as third-party services evolve — for example, Cloudinary SDK version changes, MongoDB Atlas connection string updates, and JWT library security patches. The modular project structure ensures such updates are isolated to specific configuration files.

4.6.3 Perfective Maintenance

Perfective maintenance involves adding enhancements such as email notifications on application status change using Nodemailer, server-side pagination for large job listings, improved real-time search filters, and a Redis caching layer to reduce repeated MongoDB reads on the jobs listing endpoint.

4.6.4 Preventive Maintenance

Preventive maintenance involves regularly rotating JWT secrets, monitoring Cloudinary storage usage against the free-tier limit (25 GB), backing up the MongoDB Atlas database, and reviewing dependency versions for security vulnerabilities using npm audit.